

Trabajo Práctico 1

Registro de préstamos de la biblioteca

Padrón	115569
Nombre y apellido	Dante Luca Ortega
Email	danteluca06@gmail.com

1. Informe del trabajo realizado

1.1 Objetivo

El objetivo del trabajo es construir un programa de consola que procese el archivo de préstamos de la biblioteca de la facultad, valide cada línea, calcule días de atraso y multas por socio, y genere dos reportes de salida (uno en formato de texto y otro en CSV), además de un ranking de los títulos más pedidos. El TP pone en práctica los conceptos de Tipo de Dato Abstracto (TDA), separación entre contrato e implementación, validación de invariantes en el constructor, manejo de excepciones propias y lectura/escritura de archivos con `java.nio.file.Files`.

1.2 Estructura de la solución

Todas las clases viven en el paquete `ar.uba.fi.cb100.clases.a2026.c02.tps.tp1`. Se implementaron los 6 tipos pedidos por el enunciado, cada uno en su propio archivo, más las clases auxiliares necesarias para la excepción propia y el resultado de la carga:

Clase / Interfaz	Responsabilidad
Prestamo (record)	Clase de valor. Valida sus invariantes en el constructor compacto y calcula vencimiento, días de atraso y multa.
RegistroDePrestamos (interfaz)	Contrato del TDA: qué operaciones soporta un registro de préstamos, sin decir cómo se guardan.
RegistroSobreArreglo	Implementación del contrato sobre un arreglo interno que crece a mano (capacidad inicial 8, se duplica al llenarse).
LineaInvalidaException	Excepción propia (<code>RuntimeException</code>) para representar una línea del archivo con datos inválidos, con su número de línea real.
ResultadoDeCarga (record)	Agrupar el registro cargado, los errores detectados y la cantidad de líneas de datos procesadas.
LectorDePrestamos	Lee el archivo con <code>Files.readAllLines</code> , ignora blancos y comentarios, arma un <code>Prestamo</code> por línea válida y captura los errores sin abortar.
FilaDeSocio (record)	Fila consolidada del reporte: totales de préstamos, atraso, multa y estado de un socio.
Reporteador	Arma las filas del reporte (<code>porSocio</code>) y el ranking de títulos (<code>ranking</code>) a partir de un <code>RegistroDePrestamos</code> .
ExportadorDeReporte (interfaz)	Contrato de exportación: <code>exportar(filas, destino)</code> y <code>extension()</code> .
ExportadorTxt ExportadorCsv	Implementaciones concretas del exportador, una por formato de salida.
Tp1	Contiene el <code>main()</code> : orquesta lectura, reporte, exportación e impresión del resumen por consola.

1.3 Reglas de negocio implementadas

- El plazo de préstamo es de 14 días corridos desde el retiro (`Prestamo.vencimiento()`).
- Un préstamo sin fecha de devolución está pendiente (`estaPendiente()`).
- Los días de atraso se cuentan desde el vencimiento hasta la devolución (o hasta la fecha de corte si está pendiente), nunca un valor negativo.
- La multa es de \$150 por día de atraso, con un tope de \$3000 por préstamo.
- Un socio queda `CON_DEUDA` si su multa acumulada es mayor a 0; si no, `AL_DIA`.

1.4 Manejo de errores de formato

El lector no aborta la ejecución ante una línea inválida: la descarta, guarda el motivo junto con el número de línea real del archivo (contando comentarios y blancos) en un arreglo de errores que se redimensiona a mano, y continúa con las siguientes líneas. Se detectan cuatro tipos de error: cantidad incorrecta de campos, fecha inválida, padrón no numérico y devolución anterior al retiro (esta última delegada a la validación del constructor de Prestamo).

1.5 Restricciones respetadas

- No se usó ninguna colección de `java.util` (`ArrayList`, `HashMap`, `HashSet`): todo se resolvió con arreglos, usando `Arrays.copyOf` para el crecimiento manual.
- No se usó la API de Streams.
- No se usaron librerías externas (no se implementó el punto adicional de Excel).
- Se usaron rutas relativas para los archivos de entrada y salida.

1.6 Resultado obtenido con el archivo de prueba

Con el archivo `datos/prestamos.csv` entregado y la fecha de corte por defecto (2026-05-04), el programa procesa 22 líneas de datos, registra 18 préstamos válidos y descarta 4 líneas inválidas (una de cada tipo de error pedido). El resumen impreso por consola es:

```
Lineas de datos: 22 | validas: 18 | descartadas: 4
linea 28: se esperaban 6 campos y llegaron 4
linea 29: fecha invalida: 2026-13-02
linea 30: padron no numerico: CUARENTA
linea 31: la devolucion (2026-04-01) es anterior al retiro (2026-04-15)
```

El reporte consolidado por socio (reporte.txt / reporte.csv) da:

Padrón	Socio	Préstamos	Días atraso	Multa	Estado
39876	Bruno Ferrari	4	24	3600	CON_DEUDA
41234	Ana Gomez	4	23	3450	CON_DEUDA
40555	Diego Ruiz	3	42	3300	CON_DEUDA
43310	Elena Sosa	3	10	1500	CON_DEUDA
42001	Carla Nunez	4	0	0	AL_DIA
TOTALES		18	99	11850	

Y el ranking de títulos más pedidos:

```
TITULOS MAS PEDIDOS
1. Estructuras de Datos      6
2. Clean Code                3
3. Introduction to Algorithms 3
```

Clean Code e Introduction to Algorithms empatan en 3 préstamos; el desempate se resuelve alfabéticamente, por lo que Clean Code queda primero.

1.7 Pruebas automáticas

Se entregan pruebas con JUnit 5 que cubren, como mínimo: las validaciones del constructor de Prestamo (`assertThrows` para cada invariante), el cálculo de vencimiento, atraso y tope de multa; el crecimiento de `RegistroSobreArreglo` más allá de su capacidad inicial; `padrones()` sin repetidos;

prestamosDe() con un padrón inexistente; el desempate alfabético del ranking; la carga del archivo de prueba (18 válidas / 4 errores); y la exportación a CSV verificada escribiendo a un archivo temporal y releyéndolo con Files.readAllLines.

2. Cuestionario

1) ¿Qué es un TDA? ¿Por qué conviene separar el contrato (la interfaz) de la implementación?

Un Tipo de Dato Abstracto (TDA) es un conjunto de valores junto con las operaciones que se pueden hacer sobre ellos, definido exclusivamente por **qué hace** cada operación (su comportamiento observable), sin especificar **cómo** se implementa internamente. El contrato es la interfaz pública que ve quien usa el TDA; la implementación es un detalle oculto de representación y algoritmos.

Separar el contrato de la implementación conviene porque reduce el acoplamiento: el código cliente (por ejemplo Reporteador o Tp1) programa contra RegistroDePrestamos y no le importa si por dentro hay un arreglo, una lista enlazada o un árbol. Esto permite cambiar la implementación sin tocar el resto del programa (como se ve con ExportadorDeReporte: agregar un tercer formato de exportación no requiere modificar Tp1), facilita el testing contra el contrato, y protege las invariantes internas de la implementación de manipulaciones externas.

2) ¿Qué es una invariante de clase? ¿Por qué conviene validarla en el constructor y no al principio de cada método?

Una invariante de clase es una condición que debe cumplirse siempre para cualquier instancia válida de esa clase, en todo momento en que un objeto es observable desde afuera. En Prestamo, por ejemplo: el padrón siempre es positivo, socio/isbn/titulo nunca son nulos ni vacíos, y si hay fecha de devolución, no es anterior a la de retiro.

Conviene validarla en el constructor porque así se garantiza que jamás puede existir en memoria una instancia mal formada: el objeto nace válido o directamente no nace (fail-fast). Si en cambio se validara al principio de cada método, habría que repetir la misma validación en todos los métodos que usan esos datos, con el riesgo de olvidarse de alguno, y encima existiría una ventana de tiempo en la que un objeto inconsistente circula por el programa. Validar una sola vez, en el único punto de creación, es más simple y más seguro.

3) Su implementación duplica la capacidad del arreglo cada vez que se llena. Al registrar N préstamos, ¿cuántas copias de elementos se hacen en total? ¿Qué pasaría si en lugar de duplicar, el arreglo creciera de a un lugar por vez?

Con capacidad inicial 8 y factor de crecimiento 2, las redimensiones ocurren en tamaños $8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow \dots$, y cada redimensión de tamaño k a $2k$ copia k elementos. Sumando todas las copias hasta alcanzar N elementos se obtiene una serie geométrica cuyo total es menor a $2N$ (aproximadamente $2N$ en el peor caso). Es decir, el costo total de copiar elementos crece linealmente en N , aunque una llamada puntual a registrar() pueda costar $O(N)$ cuando justo toca redimensionar: el costo **amortizado** de registrar() es $O(1)$.

Si en cambio el arreglo creciera de a un lugar por vez (capacidad+1 en cada resize), al insertar el i -ésimo elemento que provoca overflow habría que copiar los $i-1$ elementos ya existentes. Sumando desde 1 hasta N se obtiene $0+1+2+\dots+(N-1) = N \cdot (N-1)/2$, es decir el total de copias pasa a ser $O(N^2)$. Cada llamada a registrar() seguiría costando $O(N)$ en el peor caso, pero ahora eso sucedería en **cada** inserción y no de forma amortizada, degradando el rendimiento general de la estructura de $O(1)$ amortizado a $O(N)$ por inserción.

4) ¿Qué diferencia hay entre == y equals al comparar dos objetos Prestamo? ¿Por qué al usar un record el equals funciona sin escribirlo?

El operador `==` compara identidad de referencia: es verdadero únicamente si ambas variables apuntan exactamente al mismo objeto en memoria. El método `equals()` compara igualdad de valor, según el

criterio que defina la clase: puede devolver true para dos objetos distintos en memoria si su contenido se considera equivalente.

Un record en Java genera automáticamente equals(), hashCode() y toString() a partir de **todos** sus componentes, comparándolos uno a uno (usando equals() en cada campo, y manejando null correctamente). Esto es exactamente la semántica que se busca para Prestamo como clase de valor: dos préstamos con el mismo retiro, padrón, socio, isbn, título y devolución son, a efectos del programa, el mismo préstamo, sin necesidad de escribir ese método a mano ni de mantenerlo sincronizado si se agrega o quita un campo.

5) ¿Por qué hay que usar split(";", -1) y no split(";") para partir cada línea del archivo? ¿Qué préstamos se romperían si se usara la segunda forma?

String.split(regex) sin límite (equivalente a límite 0) descarta los campos vacíos que queden al **final** del arreglo resultante. String.split(regex, -1), en cambio, conserva todos los campos, incluidos los vacíos finales.

Como el campo fechaDevolucion es el último de la línea y puede venir vacío para un préstamo pendiente, una línea como "2026-03-11;40555;Diego Ruiz;9788478290499;El Lenguaje de Programacion C;" (terminada en ; sin nada después) produciría, con split(";") sin el límite -1, solo 5 campos en lugar de 6, porque el campo vacío del final se descarta. El lector rechazaría esa línea con el error "se esperaban 6 campos y llegaron 5", pese a ser una línea perfectamente válida. En la práctica, esto rompería **todos** los préstamos que todavía están en poder del socio (sin fecha de devolución), que en el archivo de prueba son varios.

3. Manual de usuario

3.1 ¿Para qué sirve?

Este programa lee la lista de préstamos de la biblioteca, calcula quién tiene libros atrasados y cuánto tiene que pagar de multa, y arma dos planillas con esa información: una para leer o imprimir y otra para abrir con Excel. De yapa, también arma un ranking con los libros más pedidos.

3.2 Antes de arrancar

El programa no tiene una pantalla con botones para hacer click: se corre escribiendo un par de líneas en una ventana de texto (la "terminal" o "consola"). No hace falta saber programar ni entender lo que dice esa ventana — alcanza con copiar y pegar lo que se indica más abajo y apretar Enter.

3.3 Cómo hacerlo andar

- Abrí la carpeta del proyecto. Adentro, abrí una terminal en esa misma carpeta (en Windows: click derecho dentro de la carpeta → "Abrir en Terminal"; en Mac: buscá la aplicación "Terminal" y arrastrá la carpeta adentro de la ventana que se abre).
- Escribí (o copió y pegá) lo siguiente y apretá Enter:

```
./gradlew run
```

La primera vez puede tardar un ratito más de lo normal porque el programa descarga algunas cosas de internet; después arranca rápido. Cuando termina, en la misma ventana aparece un resumen de cuántos préstamos se cargaron bien y cuáles tuvieron algún problema.

Si en algún momento querés usar otro archivo de préstamos, o calcular las multas a otra fecha, se lo podés pedir agregando esos datos al final (esto es opcional; si no se los das, usa el archivo y la fecha de ejemplo):

```
./gradlew run --args="miArchivo.csv 2026-06-01"
```

3.4 Cómo tiene que estar armado el archivo de préstamos

Es un archivo de texto simple, de los que se pueden abrir hasta con el Bloc de notas. Cada línea es un préstamo, y los datos van separados por punto y coma (;), siempre en este orden: fecha en que se retiró el libro, número de socio, nombre del socio, ISBN, título del libro, y fecha en que lo devolvió. Si todavía no lo devolvió, ese último dato se deja vacío.

```
2026-03-02;41234;Ana Gomez;9789871234567;Estructuras de Datos;2026-03-16
```

- Las fechas se escriben como año-mes-día (por ejemplo, 2 de marzo de 2026 es 2026-03-02).
- Si querés dejar alguna línea como anotación o comentario, hacela empezar con #: el programa la salta y no la cuenta como un error.
- Las líneas vacías también se saltan sin problema.

3.5 Qué te vas a encontrar después de correrlo

En la misma ventana donde escribiste el comando aparece un resumen: cuántos préstamos había en total, cuántos se pudieron cargar sin problemas y cuántos tuvieron algún error (con el detalle de cuál fue el problema en cada caso).

Además, en esa misma carpeta van a aparecer dos archivos nuevos:

- `reporte.txt`: el listado de multas por socio, de mayor a menor deuda, con el ranking de los libros más pedidos al final. Se puede abrir con cualquier editor de texto.
- `reporte.csv`: la misma información pero pensada para abrirse con Excel — simplemente hacele doble click.

3.6 ¿Y si algo sale mal?

Tranquilo: si alguna línea del archivo tiene un error (por ejemplo, le falta un dato o la fecha está mal escrita), el programa no se rompe ni se cuelga. Simplemente avisa cuál es el problema y en qué línea está, se salta esa línea, y sigue procesando todo lo demás con normalidad. Si el archivo que le pediste directamente no existe, te va a avisar con un mensaje indicando que no lo pudo encontrar.

4. Manual del programador

4.1 Organización del código

Todo el código vive en un único paquete: `ar.uba.fi.cb100.clases.a2026.c02.tps.tp1`. No hay subpaquetes: al tratarse de un TP acotado, se priorizó la simplicidad sobre una estructura en capas más elaborada.

4.2 Prestamo — clase de valor

Es un record con seis componentes. El constructor compacto valida todas las invariantes (fechas, padrón positivo, textos no vacíos, coherencia entre retiro y devolución) y lanza `IllegalArgumentException` ante cualquier violación, de forma que nunca puede existir un `Prestamo` mal formado. Expone `estaPendiente()`, `vencimiento()`, `diasDeAtraso(corte)` y `multa(corte)`; estos dos últimos son los únicos puntos donde vive la lógica de negocio de atraso y multa, evitando duplicarla en el resto del programa.

4.3 RegistroDePrestamos / RegistroSobreArreglo

`RegistroDePrestamos` es la interfaz (el contrato del TDA): registrar, cantidad, obtener, padrones, `prestamosDe` y `titulosMasPedidos`. `RegistroSobreArreglo` es la única implementación actual, sobre un arreglo `Prestamo[]` con capacidad inicial 8 que se duplica (`Arrays.copyOf`) cuando se llena. `cantidad()` siempre refleja los elementos usados, no la capacidad interna del arreglo. Para agregar una segunda implementación (por ejemplo, sobre lista enlazada) alcanza con implementar la interfaz; ningún otro código del proyecto depende de la representación interna.

4.4 Lectura y validación: LectorDePrestamos

`cargar(Path)` lee todas las líneas con `Files.readAllLines`, ignora blancos y comentarios (`#`), y por cada línea de datos intenta construir un `Prestamo`. Los errores de formato (cantidad de campos, fechas, padrón no numérico) se detectan explícitamente antes de delegar en el constructor de `Prestamo`; los errores de invariante de negocio (por ejemplo, devolución anterior al retiro) se detectan porque el constructor de `Prestamo` tira `IllegalArgumentException`, que se recaptura y se re-envuelve como `LineaInvalidaException` junto con el número de línea real del archivo. El arreglo de errores se redimensiona a mano con la misma estrategia de duplicación que `RegistroSobreArreglo`.

4.5 Reportes: Reporteador y ExportadorDeReporte

`Reporteador.porSocio(registro, corte)` recorre los padrones únicos, suma préstamos/atraso/multa de cada socio y arma un `FilaDeSocio` por padrón, ordenado por multa descendente (y alfabéticamente por socio en caso de empate). `Reporteador.ranking(registro, n)` delega en `titulosMasPedidos(n)` del registro para obtener el orden y agrega, por cada título, la cantidad de préstamos asociados. La exportación a disco está detrás de la interfaz `ExportadorDeReporte` (`exportar + extension`), con dos implementaciones concretas (`ExportadorTxt` y `ExportadorCsv`). `Tp1` solo conoce el contrato `ExportadorDeReporte`, nunca la implementación concreta que está usando; agregar un tercer formato de exportación no requiere tocar `Tp1` ni `Reporteador`.

4.6 Punto de entrada: Tp1

`main(args)` resuelve el archivo de entrada y la fecha de corte (con valores por defecto si no se pasan argumentos), delega la carga en `LectorDePrestamos`, imprime el resumen por consola, arma las filas del reporte con `Reporteador` y exporta con ambos `ExportadorDeReporte`. Todas las rutas usadas son relativas, tal como pide el enunciado.

4.7 Algoritmos y complejidad

- **Crecimiento del arreglo:** duplicación de capacidad; costo amortizado $O(1)$ por inserción (ver cuestionario, pregunta 3).
- **padrones() / prestamosDe():** búsqueda lineal sobre el arreglo interno; $O(n)$ y $O(n)$ respectivamente, donde n es la cantidad de préstamos registrados.
- **titulosMasPedidos(n):** primero cuenta ocurrencias por título con búsqueda lineal ($O(n \cdot t)$, con t = títulos distintos), luego ordena por selección ($O(t^2)$), aceptable para el tamaño de datos del TP.
- **Reporteador.porSocio:** ordena las filas resultantes con burbuja ($O(s^2)$, s = cantidad de socios distintos).

4.8 Cómo correr los tests

```
./gradlew test
```

Las pruebas están en el mismo paquete, bajo el árbol de test de Gradle, y cubren Prestamo, RegistroSobreArreglo, LectorDePrestamos y ExportadorCsv, tal como pide el enunciado.